



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт Информационных технологий

Кафедра информационных технологий в атомной энергетике

ОТЧЕТ ПО ПРАКТИКЕ №12

«Автоматизированная система управления для обеспечения инвентарем для лыжного спорта»

по дисциплине «Автоматизированные системы управления элементами
промышленных и административных объектов»

Студент группы ИКБО-50-23

Павлов Н.С.

(подпись студента)

Принял преподаватель

Щербаков К. В.

(подпись руководителя)

Работа представлена

« ____ » _____ 2026 г.

Допущен к работе

« ____ » _____ 2026 г.

Москва 2026

ОГЛАВЛЕНИЕ

1. Обоснование выбора технологий	3
2. Архитектура модуля и структура проекта	4
3. Описание функциональности	5
3.1. Дашборд	5
3.2. Спортсмены.....	6
3.3. Выдачи.....	6
3.4. Склад	7
3.5. Статистика	7
4. Исходный код	9
5. Результаты работы модуля.....	33

1. Обоснование выбора технологий

Для разработки программного модуля с веб-интерфейсом, позволяющего работать с базой данных АСУ МТО, были выбраны следующие технологии:

Таблица 1.1 – Бэкенд: Python 3.12 + FastAPI

Критерий	Обоснование
Готовый код	Переиспользуется код database.py и config.py из модуля прогнозирования, что обеспечивает единство архитектуры
Созданная БД	Работает с той же БД PostgreSQL (схема asu_inventory, 19 таблиц), созданной ранее
Производительность	FastAPI — один из самых быстрых Python-фреймворков, поддерживает асинхронную обработку запросов
Документация	Автоматическая генерация Swagger UI (/docs) для тестирования API
Простота	Минимум кода для создания полноценного REST API

Таблица 1.2 – Фронтенд: HTML + CSS + JavaScript (Fetch API)

Критерий	Обоснование
Независимость	Не требует установки Node.js, npm или других инструментов сборки
Динамичность	Fetch API позволяет выполнять асинхронные запросы к бэкенду без перезагрузки страницы
Совместимость	Работает в любом современном браузере

2. Архитектура модуля и структура проекта

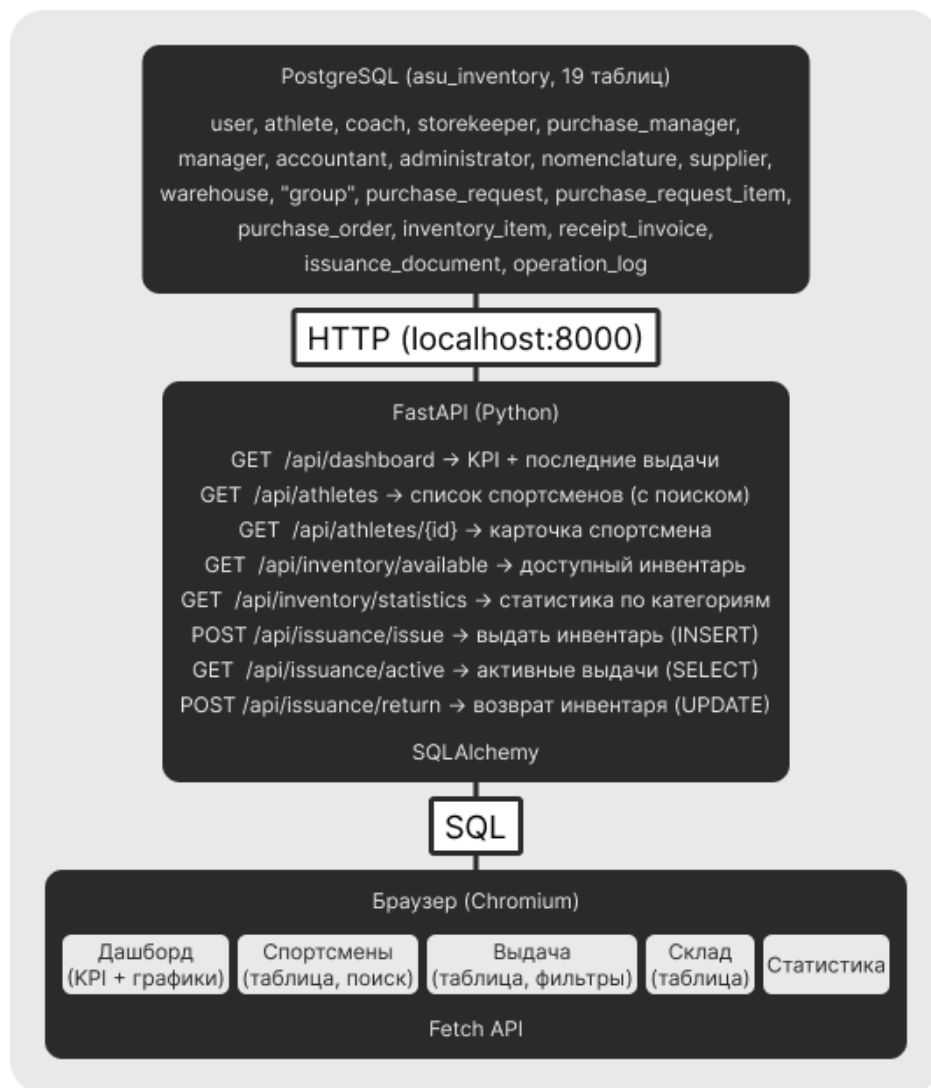


Рисунок 2.1 – Архитектура модуля

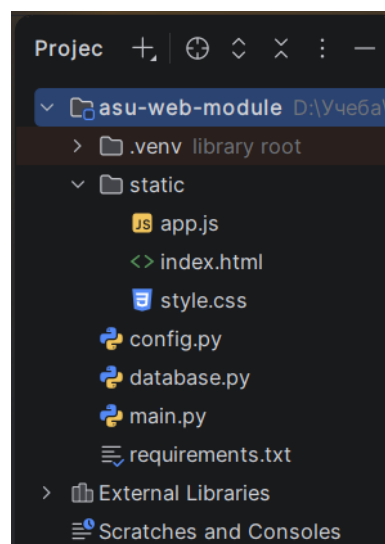


Рисунок 2.2 – Структура проекта

3. Описание функциональности

Модуль предоставляет веб-интерфейс с 5 вкладками, каждая из которых реализует определённый набор операций с базой данных.

3.1. Дашборд

Назначение: Отображение ключевых показателей системы в реальном времени.

API-эндпоинт: *GET /api/dashboard*

Выполняемые SQL-запросы:

- *SELECT COUNT(*) FROM athlete* — количество спортсменов
- *SELECT COUNT(*) FROM inventory_item WHERE status = 'AVAILABLE'* — доступный инвентарь
- *SELECT COUNT(*) FROM issuance_document WHERE status IN (...)* — активные выдачи
- *SELECT ... WHERE planned_return_date < CURRENT_DATE* — просроченные
- *SELECT ... GROUP BY nomenclature_category* — распределение по категориям
- *SELECT ... ORDER BY issuance_date DESC LIMIT 5* — последние выдачи
- *SELECT ... GROUP BY athlete ... ORDER BY overdue_items DESC LIMIT 5* — топ должников

Интерфейс:

- 5 KPI-карточек (спортсменов, доступно, выдач, просрочено, должников)
- График распределения инвентаря по категориям (CSS-бары)
- Таблица «Последние выдачи»
- Таблица «Топ должников»

3.2. Спортсмены

Назначение: Просмотр списка спортсменов с возможностью поиска и фильтрации.

API-эндпоинты:

- *GET /api/athletes?search=&has_debt=* — список с фильтрацией
- *GET /api/athletes/{id}* — детальная карточка

Выполняемые SQL-запросы:

- *SELECT ... FROM athlete JOIN "user" JOIN "group" JOIN coach WHERE full_name ILIKE ...* — поиск
- *SELECT ... WHERE athlete_id = :id* — карточка с email, телефоном, статистикой

Интерфейс:

- Поисковая строка с дебаунсом (300 мс)
- Фильтр по задолженности
- Таблица с подсветкой должников
- Кнопка просмотра детальной карточки (модальное окно)

3.3. Выдачи

Назначение: Управление выдачей и возвратом инвентаря.

API-эндпоинты:

- *GET /api/issuance/active?filter_status=* — активные выдачи
- *POST /api/issuance/issue* — выдача инвентаря (INSERT + UPDATE)
- *POST /api/issuance/return* — возврат инвентаря (UPDATE)

Выполняемые SQL-запросы:

- *SELECT ... FROM issuance_document JOIN athlete JOIN inventory_item JOIN nomenclature* — список выдач
- *INSERT INTO issuance_document (...) VALUES (...)* — создание документа

- *UPDATE inventory_item SET status = 'ISSUED'* — изменение статуса
- *UPDATE issuance_document SET status = 'RETURNED'* — возврат

Бизнес-логика:

- При выдаче проверяется, есть ли у спортсмена задолженность
- При выдаче проверяется, доступен ли инвентарь
- При возврате проверяется, не возвращён ли уже инвентарь

Интерфейс:

- Кнопка «Выдать инвентарь» → модальное окно с формой
- Выпадающие списки спортсменов и доступного инвентаря
- Таблица активных выдач с фильтром (все / просроченные / активные)
- Кнопка «Возврат» в каждой строке

3.4. Склад

Назначение: Просмотр доступного инвентаря.

API-эндпоинт: *GET /api/inventory/available*

Выполняемый SQL-запрос:

- *SELECT ... FROM inventory_item JOIN nomenclature WHERE status = 'AVAILABLE'*

Интерфейс:

- Таблица со всеми доступными единицами инвентаря

3.5. Статистика

Назначение: Анализ распределения инвентаря по категориям.

API-эндпоинт: *GET /api/inventory/statistics*

Выполняемый SQL-запрос:

- *SELECT ... GROUP BY nomenclature_category* — с разбивкой по статусам

Интерфейс:

- Карточки по категориям с числовыми показателями
- Визуализация пропорций через цветные полосы (доступен / выдан / проблемы)

4. Исходный код

Листинг 4.1 – requirements.txt

```
fastapi==0.108.0
uvicorn[standard]==0.25.0
sqlalchemy==2.0.25
psycopg2-binary==2.9.9
loguru==0.7.2
pydantic==2.5.0
python-dotenv==1.0.0
```

Листинг 4.2 – config.py

```
import os
from dataclasses import dataclass, field

@dataclass
class DatabaseConfig:
    host: str = field(default_factory=lambda: os.getenv("DB_HOST", "localhost"))
    port: int = field(default_factory=lambda: int(os.getenv("DB_PORT", "5432")))
    database: str = field(default_factory=lambda: os.getenv("DB_NAME", "postgres"))
    schema: str = field(default_factory=lambda: os.getenv("DB_SCHEMA", "asu_inventory"))
    username: str = field(default_factory=lambda: os.getenv("DB_USERNAME", "postgres"))
    password: str = field(default_factory=lambda: os.getenv("DB_PASSWORD", "CKu9LmMo"))

    @property
    def connection_string(self) -> str:
        return f"postgresql://{self.username}:{self.password}@{self.host}:{self.port}/{self.database}"

@dataclass
class AppConfig:
    db: DatabaseConfig = field(default_factory=DatabaseConfig)

config = AppConfig()
```

Листинг 4.3 – database.py

```
from sqlalchemy import create_engine
from loguru import logger
from config import config

class DatabaseManager:
    def __init__(self):
        self.engine = create_engine(
            config.db.connection_string,
            pool_size=5,
            max_overflow=10,
            echo=False,
            connect_args={'options': f'-c search_path={config.db.schema},public'}
        )
```

```
)
    logger.info(f"Подключение к БД: {config.db.host}:{config.db.port}/{config.db.database}")
```

Листинг 4.4 – main.py

```
from fastapi import FastAPI, HTTPException, Query
from fastapi.staticfiles import StaticFiles
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel
from typing import Optional, List
from datetime import date, datetime

from database import DatabaseManager
from config import config

app = FastAPI(title="ACY MTO Web API", version="2.0.0")

app.add_middleware(CORSMiddleware, allow_origins=["*"], allow_methods=["*"],
allow_headers=["*"])

db = DatabaseManager()

# ===== Pydantic-схемы =====

class AthleteResponse(BaseModel):
    athlete_id: int
    full_name: str
    sports_category: Optional[str]
    birth_year: Optional[int]
    group_name: Optional[str]
    coach_name: Optional[str]
    has_active_debt: bool

class IssueRequest(BaseModel):
    athlete_id: int
    inventory_item_id: int
    planned_return_date: date

class ReturnRequest(BaseModel):
    issuance_id: int

class MessageResponse(BaseModel):
    status: str
    message: str

# ===== ДАШБОРД =====

@app.get("/api/dashboard")
async def get_dashboard():
    """Ключевые метрики, последние выдачи, топ-должники."""
    with db.engine.connect() as conn:
        from sqlalchemy import text

        athletes_count = conn.execute(text("SELECT COUNT(*) FROM athlete")).scalar()
        available_items = conn.execute(text(
            "SELECT COUNT(*) FROM inventory_item WHERE inventory_status = 'AVAILABLE'"
        )).scalar()
        active_issuances = conn.execute(text(
```

```

        "SELECT COUNT(*) FROM issuance_document WHERE issuance_document_status IN ('ACTIVE', 'EXTENDED', 'OVERDUE')")
    ).scalar()
    overdue_count = conn.execute(text("""
        SELECT COUNT(*) FROM issuance_document
        WHERE planned_return_date < CURRENT_DATE
        AND issuance_document_status IN ('ACTIVE', 'OVERDUE')
    """)).scalar()
    debtors = conn.execute(text(
        "SELECT COUNT(*) FROM athlete WHERE has_active_debt = TRUE"
    )).scalar()

    categories = conn.execute(text("""
        SELECT n.nomenclature_category, COUNT(*) as cnt
        FROM inventory_item i
        JOIN nomenclature n ON i.inventory_item_nomenclature_id = n.nomenclature_id
        WHERE i.inventory_status = 'AVAILABLE'
        GROUP BY n.nomenclature_category ORDER BY cnt DESC
    """)).fetchall()

    recent_issuances = conn.execute(text("""
        SELECT u.full_name AS athlete_name, n.nomenclature_name AS product_name,
            d.issuance_date, d.planned_return_date,
            CASE WHEN d.planned_return_date < CURRENT_DATE THEN
'OVERDUE' ELSE 'ACTIVE' END AS effective_status
        FROM issuance_document d
        JOIN athlete a ON d.issuance_document_athlete_id = a.athlete_id
        JOIN "user" u ON a.athlete_user_id = u.user_id
        JOIN inventory_item i ON d.issuance_document_inventory_item_id = i.inventory_item_id
        JOIN nomenclature n ON i.inventory_item_nomenclature_id = n.nomenclature_id
        WHERE d.issuance_document_status IN ('ACTIVE', 'EXTENDED', 'OVERDUE')
        ORDER BY d.issuance_date DESC LIMIT 5
    """)).fetchall()

    top_debtors = conn.execute(text("""
        SELECT u.full_name AS athlete_name, a.sports_category,
            COUNT(d.issuance_document_id) AS overdue_items,
            MAX(CURRENT_DATE - d.planned_return_date) AS max_days
        FROM athlete a
        JOIN "user" u ON a.athlete_user_id = u.user_id
        JOIN issuance_document d ON a.athlete_id = d.issuance_document_athlete_id
        WHERE d.planned_return_date < CURRENT_DATE
        AND d.issuance_document_status IN ('ACTIVE', 'OVERDUE')
        GROUP BY u.full_name, a.sports_category
        ORDER BY overdue_items DESC LIMIT 5
    """)).fetchall()

    return {
        "athletes_count": athletes_count,
        "available_items": available_items,
        "active_issuances": active_issuances,
        "overdue_count": overdue_count,
        "debtors": debtors,
        "categories": [{"category": r[0], "count": r[1]} for r in categories],
        "recent_issuances": [dict(r._mapping) for r in recent_issuances],
        "top_debtors": [dict(r._mapping) for r in top_debtors]
    }

```

```

    }

# ===== СПОРТСМЕНЫ =====

@app.get("/api/athletes")
async def get_athletes(
    search: str = Query("", description="Поиск по фамилии"),
    has_debt: Optional[str] = Query(None, description="Фильтр по задолженно-
сти")
):
    query = """
    SELECT
        a.athlete_id, u.full_name, a.sports_category, a.birth_year,
        g.group_name, cu.full_name AS coach_name, a.has_active_debt,
        (SELECT COUNT(*) FROM issuance_document d
         WHERE d.issuance_document_athlete_id = a.athlete_id
         AND d.issuance_document_status IN ('ACTIVE','EXTENDED','OVERDUE'))
    AS active_items
    FROM athlete a
    JOIN "user" u ON a.athlete_user_id = u.user_id
    LEFT JOIN "group" g ON a.athlete_group_id = g.group_id
    LEFT JOIN coach c ON a.athlete_coach_id = c.coach_id
    LEFT JOIN "user" cu ON c.coach_user_id = cu.user_id
    WHERE 1=1
    """
    params = {}
    if search:
        query += " AND u.full_name ILIKE :search"
        params["search"] = f"%{search}%"
    if has_debt == "true":
        query += " AND a.has_active_debt = TRUE"
    elif has_debt == "false":
        query += " AND a.has_active_debt = FALSE"
    query += " ORDER BY u.full_name"

    with db.engine.connect() as conn:
        from sqlalchemy import text
        result = conn.execute(text(query), params)
        return [dict(row._mapping) for row in result]

@app.get("/api/athletes/{athlete_id}")
async def get_athlete_detail(athlete_id: int):
    """Детальная информация о спортсмене."""
    query = """
    SELECT
        a.athlete_id, u.full_name, u.email, u.phone,
        a.sports_category, a.birth_year, a.has_active_debt,
        g.group_name, cu.full_name AS coach_name,
        (SELECT COUNT(*) FROM issuance_document d
         WHERE d.issuance_document_athlete_id = a.athlete_id
         AND d.issuance_document_status IN ('ACTIVE','EXTENDED','OVERDUE'))
    AS active_items,
        (SELECT COUNT(*) FROM issuance_document d
         WHERE d.issuance_document_athlete_id = a.athlete_id
         AND d.issuance_document_status = 'RETURNED') AS returned_items
    FROM athlete a
    JOIN "user" u ON a.athlete_user_id = u.user_id
    LEFT JOIN "group" g ON a.athlete_group_id = g.group_id
    LEFT JOIN coach c ON a.athlete_coach_id = c.coach_id
    LEFT JOIN "user" cu ON c.coach_user_id = cu.user_id
    WHERE a.athlete_id = :athlete_id
    """

```

```

        with db.engine.connect() as conn:
            from sqlalchemy import text
            result = conn.execute(text(query), {"athlete_id": athlete_id})
            row = result.fetchone()
            if row:
                return dict(row._mapping)
            raise HTTPException(status_code=404, detail="Спортсмен не найден")

# ===== ИМБЕХТАРЬ =====

@app.get("/api/inventory/available")
async def get_available_inventory():
    query = """
        SELECT
            i.inventory_item_id, i.barcode, i.size,
            n.nomenclature_name, n.nomenclature_category
        FROM inventory_item i
        JOIN nomenclature n ON i.inventory_item_nomenclature_id = n.nomenclature_id
        WHERE i.inventory_status = 'AVAILABLE'
        ORDER BY n.nomenclature_name, i.size
    """
    with db.engine.connect() as conn:
        from sqlalchemy import text
        result = conn.execute(text(query))
        return [dict(row._mapping) for row in result]

@app.get("/api/inventory/statistics")
async def get_inventory_statistics():
    query = """
        SELECT
            n.nomenclature_category,
            COUNT(*) FILTER (WHERE i.inventory_status = 'AVAILABLE') AS available,
            COUNT(*) FILTER (WHERE i.inventory_status = 'ISSUED') AS issued,
            COUNT(*) FILTER (WHERE i.inventory_status IN ('DEFECTIVE', 'UNDER_REPAIR')) AS problems,
            COUNT(*) AS total
        FROM inventory_item i
        JOIN nomenclature n ON i.inventory_item_nomenclature_id = n.nomenclature_id
        GROUP BY n.nomenclature_category ORDER BY total DESC
    """
    with db.engine.connect() as conn:
        from sqlalchemy import text
        result = conn.execute(text(query))
        return [dict(row._mapping) for row in result]

# ===== ВЫДАЧА =====

@app.post("/api/issuance/issue", response_model=MessageResponse)
async def issue_inventory(request: IssueRequest):
    try:
        with db.engine.connect() as conn:
            from sqlalchemy import text

            result = conn.execute(
                text("SELECT has_active_debt FROM athlete WHERE athlete_id = :id"),
                {"id": request.athlete_id}
            )

```

```

        row = result.fetchone()
        if not row:
            raise HTTPException(status_code=404, detail="Спортсмен не найден")

        if row[0]:
            return MessageResponse(status="error", message="⚠ Спортсмен имеет задолженность! Выдача невозможна.")

        result = conn.execute(
            text("SELECT inventory_status FROM inventory_item WHERE inventory_item_id = :id"),
            {"id": request.inventory_item_id}
        )
        row = result.fetchone()
        if not row or row[0] != 'AVAILABLE':
            return MessageResponse(status="error", message="⚠ Инвентарь недоступен для выдачи.")

        result = conn.execute(
            text("""
                INSERT INTO issuance_document
                (issuance_document_inventory_item_id, issuance_document_athlete_id,
                 issuance_document_storekeeper_id, issuance_date,
                 planned_return_date,
                 issuance_document_status, condition_on_issue)
                VALUES (:inv_id, :ath_id, 1, CURRENT_DATE, :ret_date,
                'ACTIVE', 'NEW')
                RETURNING issuance_document_id
            """),
            {"inv_id": request.inventory_item_id, "ath_id": request.athlete_id, "ret_date": request.planned_return_date}
        )
        issuance_id = result.scalar()

        conn.execute(
            text("""
                UPDATE inventory_item
                SET inventory_status = 'ISSUED', current_holder_id = :ath_id
                WHERE inventory_item_id = :inv_id
            """),
            {"ath_id": request.athlete_id, "inv_id": request.inventory_item_id}
        )
        conn.commit()

        return MessageResponse(status="success", message=f"✅ Инвентарь успешно выдан! Документ №{issuance_id}")

    except HTTPException:
        raise
    except Exception as e:
        return MessageResponse(status="error", message=str(e))

# ===== АКТИВНЫЕ ВЫДАЧИ =====

@app.get("/api/issuance/active")
async def get_active_issuances(filter_status: str = Query("all")):
    query = """
    SELECT
        d.issuance_document_id, u.full_name AS athlete_name,

```

```

        i.barcode, n.nomenclature_name AS product_name,
        i.size, d.issuance_date, d.planned_return_date,
        CASE
            WHEN d.planned_return_date < CURRENT_DATE AND d.issuance_document_status = 'ACTIVE'
            THEN 'OVERDUE'
            ELSE d.issuance_document_status
        END AS effective_status,
        CASE
            WHEN d.planned_return_date < CURRENT_DATE
            THEN (CURRENT_DATE - d.planned_return_date)
            ELSE 0
        END AS days_overdue
    FROM issuance_document d
    JOIN athlete a ON d.issuance_document_athlete_id = a.athlete_id
    JOIN "user" u ON a.athlete_user_id = u.user_id
    JOIN inventory_item i ON d.issuance_document_inventory_item_id = i.inventory_item_id
    JOIN nomenclature n ON i.inventory_item_nomenclature_id = n.nomenclature_id
    WHERE d.issuance_document_status IN ('ACTIVE', 'EXTENDED', 'OVERDUE')
    """

    params = {}
    if filter_status == "overdue":
        query += " AND d.planned_return_date < CURRENT_DATE"
    elif filter_status == "active":
        query += " AND d.planned_return_date >= CURRENT_DATE"
    query += " ORDER BY days_overdue DESC, d.issuance_date DESC"

    with db.engine.connect() as conn:
        from sqlalchemy import text
        result = conn.execute(text(query), params)
        return [dict(row._mapping) for row in result]

# ===== BO3BPAT =====

@app.post("/api/issuance/return", response_model=MessageResponse)
async def return_inventory(request: ReturnRequest):
    try:
        with db.engine.connect() as conn:
            from sqlalchemy import text

            result = conn.execute(
                text("""
                    SELECT issuance_document_inventory_item_id, issuance_document_status
                    FROM issuance_document WHERE issuance_document_id = :id
                """),
                {"id": request.issuance_id}
            )
            row = result.fetchone()
            if not row:
                return MessageResponse(status="error", message="Документ выдачи не найден.")
            if row[1] == 'RETURNED':
                return MessageResponse(status="error", message="Инвентарь уже возвращён.")

            conn.execute(
                text("""
                    UPDATE issuance_document
                    SET actual_return_date = CURRENT_DATE,
                    issuance_document_status = 'RETURNED',

```

```

        condition_on_return = 'GOOD'
        WHERE issuance_document_id = :id
        """),
        {"id": request.issuance_id}
    )

    conn.execute(
        text("""
            UPDATE inventory_item
            SET inventory_status = 'AVAILABLE', current_holder_id =
NULL
            WHERE inventory_item_id = :inv_id
            """),
        {"inv_id": row[0]}
    )
    conn.commit()

    return MessageResponse(status="success", message="✅ Инвентарь
успешно возвращён!")

except Exception as e:
    return MessageResponse(status="error", message=str(e))

# ===== СТАТИКА =====

app.mount("/", StaticFiles(directory="static", html=True), name="static")

if __name__ == "__main__":
    import uvicorn
    uvicorn.run("main:app", host="0.0.0.0", port=8000, reload=True)

```

Листинг 4.5 – static/index.html

```

<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>АСУ МТО – Управление инвентарём</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <!-- Шапка -->
    <header>
        <div class="header-content">
            <h1>🏃 АСУ МТО</h1>
            <span class="subtitle">Лыжная организация</span>
        </div>
        <nav>
            <button onclick="showTab('dashboard')" class="active">📊
Дашборд</button>
            <button onclick="showTab('athletes')">🏊 Спортсмены</button>
            <button onclick="showTab('issuances')">📄 Выдачи</button>
            <button onclick="showTab('inventory')">📦 Склад</button>
            <button onclick="showTab('statistics')">📈 Статистика</button>
        </nav>
    </header>

    <!-- Тост -->
    <div id="toast" class="toast hidden"></div>

```



```

<main>
  <!-- Дашборд -->
  <section id="tab-dashboard" class="tab active">
    <div class="kpi-grid" id="kpi-grid">
      <div class="kpi-card"><div class="kpi-value skeleton">--
</div><div class="kpi-label">Спортсменов</div></div>
      <div class="kpi-card"><div class="kpi-value skeleton">--
</div><div class="kpi-label">Доступно инвентаря</div></div>
      <div class="kpi-card"><div class="kpi-value skeleton">--
</div><div class="kpi-label">Активных выдач</div></div>
      <div class="kpi-card"><div class="kpi-value skeleton">--
</div><div class="kpi-label">Просрочено</div></div>
      <div class="kpi-card warning"><div class="kpi-value skele-
ton">--</div><div class="kpi-label">Должников</div></div>
    </div>
    <div class="dashboard-grid">
      <div class="chart-card">
        <h3><img alt="Inventory icon" data-bbox="398 312 418 328"/> Инвентарь по категориям (в наличии)</h3>
        <div id="category-chart"></div>
      </div>
      <div class="chart-card">
        <h3><img alt="Recent issues icon" data-bbox="398 368 418 384"/> Последние выдачи</h3>
        <div class="table-wrapper">
          <table>
<thead><tr><th>Спортсмен</th><th>Товар</th><th>Выдано</th><th>Вернуть
до</th><th>Статус</th></tr></thead>
          <tbody id="recent-issuances-tbody"></tbody>
        </table>
      </div>
    </div>
    <div class="chart-card">
      <h3><img alt="Debtors icon" data-bbox="398 522 418 538"/> Топ должников</h3>
      <div class="table-wrapper">
        <table>
<thead><tr><th>Спортсмен</th><th>Разряд</th><th>Просрочено</th><th>Макс.
дней</th></tr></thead>
          <tbody id="top-debtors-tbody"></tbody>
        </table>
      </div>
    </div>
  </div>
</section>

  <!-- Спортсмены -->
  <section id="tab-athletes" class="tab">
    <div class="section-header">
      <h2><img alt="Athletes icon" data-bbox="358 738 378 754"/> Спортсмены</h2>
    </div>
    <div class="controls">
      <input type="text" id="athlete-search" placeholder="🔍
Поиск по фамилии..." oninput="debounceSearch()">
      <select id="debt-filter" onchange="loadAthletes()">
        <option value="">Все</option>
        <option value="true">Должники</option>
        <option value="false">Без долгов</option>
      </select>
    </div>
    <div class="table-wrapper">
      <table>

```

```
<tr><th>ФИО</th><th>Разряд</th><th>Год</th><th>Группа</th><th>Тренер<
/th><th>Выдач</th><th>Долг</th><th></th></tr></thead>
    <tbody id="athletes-tbody"></tbody>
  </table>
</div>
</section>

<!-- Выдачи -->
<section id="tab-issuances" class="tab">
  <div class="section-header">
    <h2>📋 Выдачи</h2>
    <div class="controls">
      <button class="btn-primary" onclick="openIssue-
Modal()">📄 Выдать инвентарь</button>
      <select id="issuance-filter" onchange="loadIssuances()">
        <option value="all">Все</option>
        <option value="overdue">Просроченные</option>
        <option value="active">Активные</option>
      </select>
    </div>
  </div>
  <div class="table-wrapper">
    <table>



```

```

        <select id="issue-inventory" required><option value="">–
Сначала выберите спортсмена –</option></select>
        <label>Дата возврата</label>
        <input type="date" id="issue-return-date" required>
        <button type="submit" class="btn-primary-full">🚀 Вы-
дать</button>
    </form>
</div>
</div>

<!-- Модальное окно: Детали спортсмена -->
<div id="athlete-modal" class="modal hidden">
    <div class="modal-content">
        <span class="modal-close" onclick="closeAthleteModal()">×</span>
        <div id="athlete-detail"></div>
    </div>
</div>

<script src="app.js"></script>
</body>
</html>

```

Листинг 4.6 – static/style.css

```

/* ===== ПЕРЕМЕННЫЕ ===== */
:root {
    --primary: #1a3a5c;
    --primary-light: #2d6a9f;
    --success: #27ae60;
    --warning: #f39c12;
    --danger: #e74c3c;
    --bg: #f0f4f8;
    --card: #ffffff;
    --text: #333333;
    --text-light: #666666;
    --radius: 12px;
    --shadow: 0 4px 20px rgba(0, 0, 0, 0.08);
}

/* ===== БАЗОВЫЕ СТИЛИ ===== */
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}

body {
    font-family: 'Segoe UI', 'Arial', sans-serif;
    background: var(--bg);
    color: var(--text);
    min-height: 100vh;
}

/* ===== ШАПКА ===== */
header {
    background: linear-gradient(135deg, #0f2640, var(--primary), var(--primary-light));
    color: white;
    padding: 20px 30px;
    display: flex;
    justify-content: space-between;
    align-items: center;
    flex-wrap: wrap;
}

```

```

        gap: 15px;
    }

    .header-content h1 {
        font-size: 1.6em;
    }

    .header-content .subtitle {
        font-size: 0.85em;
        opacity: 0.8;
    }

    nav {
        display: flex;
        gap: 5px;
        flex-wrap: wrap;
    }

    nav button {
        background: rgba(255, 255, 255, 0.1);
        color: white;
        border: none;
        padding: 10px 18px;
        border-radius: 8px;
        cursor: pointer;
        font-size: 0.9em;
        transition: all 0.3s;
        white-space: nowrap;
    }

    nav button:hover,
    nav button.active {
        background: rgba(255, 255, 255, 0.25);
        transform: translateY(-1px);
    }

    /* ===== TOCT ===== */
    .toast {
        position: fixed;
        top: 20px;
        right: 20px;
        padding: 15px 25px;
        border-radius: var(--radius);
        color: white;
        font-weight: 500;
        z-index: 9999;
        transition: all 0.4s;
        box-shadow: var(--shadow);
    }

    .toast.success {
        background: var(--success);
    }

    .toast.error {
        background: var(--danger);
    }

    .toast.hidden {
        opacity: 0;
        transform: translateX(100%);
    }

    /* ===== KOHTEHT ===== */

```

```

main {
  max-width: 1300px;
  margin: 30px auto;
  padding: 0 25px;
}

.tab {
  display: none;
  animation: fadeIn 0.3s;
}

.tab.active {
  display: block;
}

@keyframes fadeIn {
  from {
    opacity: 0;
    transform: translateY(10px);
  }
  to {
    opacity: 1;
    transform: translateY(0);
  }
}

/* ===== KPI ===== */
.kpi-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(180px, 1fr));
  gap: 20px;
  margin-bottom: 30px;
}

.kpi-card {
  background: var(--card);
  padding: 25px;
  border-radius: var(--radius);
  box-shadow: var(--shadow);
  text-align: center;
  transition: transform 0.3s;
}

.kpi-card:hover {
  transform: translateY(-3px);
}

.kpi-card.warning {
  border-left: 4px solid var(--warning);
}

.kpi-value {
  font-size: 2.2em;
  font-weight: 700;
  color: var(--primary);
}

.kpi-value.skeleton {
  color: #ccc;
}

.kpi-label {
  color: var(--text-light);
  margin-top: 5px;
}

```

```

    font-size: 0.9em;
}

/* ===== ДАШБОРД-ГРИД ===== */
.dashboard-grid {
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 20px;
}

@media (max-width: 900px) {
    .dashboard-grid {
        grid-template-columns: 1fr;
    }
}

/* ===== ГРАФИК КАТЕГОРИЙ ===== */
.chart-card {
    background: var(--card);
    padding: 25px;
    border-radius: var(--radius);
    box-shadow: var(--shadow);
}

.chart-card h3 {
    margin-bottom: 20px;
    color: var(--primary);
}

.chart-row {
    display: flex;
    align-items: center;
    gap: 15px;
    margin-bottom: 12px;
}

.chart-label {
    width: 100px;
    text-transform: capitalize;
    font-weight: 500;
}

.chart-bar-wrapper {
    flex: 1;
    background: #e8edf2;
    border-radius: 6px;
    height: 24px;
    overflow: hidden;
}

.chart-bar {
    height: 100%;
    background: linear-gradient(90deg, var(--primary-light), var(--primary));
    border-radius: 6px;
    transition: width 0.5s;
}

.chart-value {
    width: 40px;
    text-align: right;
    font-weight: 600;
    color: var(--primary);
}

```

```

/* ===== ЗАГОЛОВКИ СЕКЦИЙ ===== */
.section-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  flex-wrap: wrap;
  gap: 15px;
  margin-bottom: 25px;
}

.section-header h2 {
  color: var(--primary);
}

.controls {
  display: flex;
  gap: 10px;
  flex-wrap: wrap;
}

.controls input,
.controls select {
  padding: 8px 15px;
  border: 2px solid #ddd;
  border-radius: 8px;
  font-size: 0.9em;
  transition: border-color 0.3s;
}

.controls input:focus,
.controls select:focus {
  border-color: var(--primary-light);
  outline: none;
}

/* ===== ТАБЛИЦЫ ===== */
.table-wrapper {
  background: var(--card);
  border-radius: var(--radius);
  box-shadow: var(--shadow);
  overflow-x: auto;
}

table {
  width: 100%;
  border-collapse: collapse;
}

th {
  background: var(--primary);
  color: white;
  padding: 14px;
  text-align: left;
  font-weight: 500;
  font-size: 0.9em;
}

td {
  padding: 12px 14px;
  border-bottom: 1px solid #eee;
}

tr:hover {

```

```

        background: #f5f8fb;
    }

    tr.row-warning {
        background: #ffffdf5;
    }

    tr.row-danger {
        background: #fff5f5;
    }

    /* ===== БЕЙДЖИ ===== */
    .badge {
        padding: 5px 12px;
        border-radius: 20px;
        font-size: 0.8em;
        font-weight: 600;
    }

    .badge-active,
    .badge-extended {
        background: #d4edff;
        color: #1a6dbb;
    }

    .badge-overdue {
        background: #ffe0e0;
        color: #c0392b;
    }

    /* ===== КНОПКИ ===== */
    .btn-primary {
        background: var(--success);
        color: white;
        border: none;
        padding: 10px 20px;
        border-radius: 8px;
        cursor: pointer;
        font-weight: 500;
        transition: all 0.3s;
    }

    .btn-primary:hover {
        background: #219a52;
        transform: translateY(-1px);
    }

    .btn-sm {
        background: var(--primary-light);
        color: white;
        border: none;
        padding: 6px 12px;
        border-radius: 6px;
        cursor: pointer;
    }

    .btn-return {
        background: #3498db;
    }

    .btn-return:hover {
        background: #2980b9;
    }

```



```

/* ===== МОДАЛЬНОЕ ОКНО ===== */
.modal {
  position: fixed;
  top: 0;
  left: 0;
  width: 100%;
  height: 100%;
  background: rgba(0, 0, 0, 0.5);
  display: flex;
  align-items: center;
  justify-content: center;
  z-index: 1000;
}

.modal.hidden {
  display: none;
}

.modal-content {
  background: white;
  padding: 35px;
  border-radius: var(--radius);
  max-width: 500px;
  width: 90%;
  position: relative;
  animation: fadeIn 0.3s;
}

.modal-close {
  position: absolute;
  top: 15px;
  right: 20px;
  font-size: 1.5em;
  cursor: pointer;
  color: #999;
}

.modal-content h2,
.modal-content h3 {
  color: var(--primary);
  margin-bottom: 20px;
}

.modal-content label {
  display: block;
  margin: 15px 0 5px;
  font-weight: 500;
  color: var(--primary);
}

.modal-content select,
.modal-content input {
  width: 100%;
  padding: 10px;
  border: 2px solid #ddd;
  border-radius: 8px;
  font-size: 1em;
}

.modal-content select:focus,
.modal-content input:focus {
  border-color: var(--primary-light);
  outline: none;
}

```

```

.btn-primary-full {
  width: 100%;
  margin-top: 20px;
  padding: 14px;
  background: var(--success);
  color: white;
  border: none;
  border-radius: 8px;
  font-size: 1.1em;
  cursor: pointer;
  transition: 0.3s;
}

.btn-primary-full:hover {
  background: #219a52;
}

/* ===== ДЕТАЛИ СПОРТСМЕНА ===== */
.detail-grid {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 12px;
}

.detail-item {
  background: var(--bg);
  padding: 10px 14px;
  border-radius: 6px;
}

.detail-label {
  display: block;
  font-size: 0.8em;
  color: var(--text-light);
  margin-bottom: 3px;
}

.text-success {
  color: var(--success);
}

.text-danger {
  color: var(--danger);
}

/* ===== СТАТИСТИКА ===== */
#stats-container {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(300px, 1fr));
  gap: 20px;
}

.stat-card {
  background: var(--card);
  padding: 20px 25px;
  border-radius: var(--radius);
  box-shadow: var(--shadow);
}

.stat-card h3 {
  color: var(--primary);
  margin-bottom: 15px;
  text-transform: capitalize;
}

```

```

}

.stat-row {
  display: flex;
  justify-content: space-between;
  margin-bottom: 8px;
}

.stat-available {
  color: var(--success);
  font-weight: 600;
}

.stat-issued {
  color: var(--primary-light);
  font-weight: 600;
}

.stat-problems {
  color: var(--danger);
  font-weight: 600;
}

.stat-bar {
  display: flex;
  height: 20px;
  border-radius: 10px;
  overflow: hidden;
  margin-top: 10px;
  background: #e8edf2;
}

.stat-bar-available {
  background: var(--success);
}

.stat-bar-issued {
  background: var(--primary-light);
}

.stat-bar-problems {
  background: var(--danger);
}

/* ===== АДАПТИВ ===== */
@media (max-width: 768px) {
  header {
    flex-direction: column;
    text-align: center;
  }

  nav {
    justify-content: center;
  }

  nav button {
    padding: 8px 12px;
    font-size: 0.8em;
  }

  .kpi-grid {
    grid-template-columns: repeat(2, 1fr);
  }
}

```

```

    .section-header {
      flex-direction: column;
      align-items: flex-start;
    }

    .detail-grid {
      grid-template-columns: 1fr;
    }

    #stats-container {
      grid-template-columns: 1fr;
    }
  }
}

```

Листинг 4.7 – static/app.js

```

const API = 'http://localhost:8000';
let searchTimeout;

// ===== НАВИГАЦИЯ =====

function showTab(name) {
  document.querySelectorAll('.tab').forEach(t => t.classList.remove('active'));
  document.querySelectorAll('nav button').forEach(b => b.classList.remove('active'));
  document.getElementById('tab-' + name).classList.add('active');
  event.target.classList.add('active');

  const loaders = {
    dashboard: loadDashboard,
    athletes: loadAthletes,
    issuances: loadIssuances,
    inventory: loadInventory,
    statistics: loadStatistics,
  };
  if (loaders[name]) loaders[name]();
}

// ===== ТОСТЫ =====

function showToast(message, type) {
  const toast = document.getElementById('toast');
  toast.textContent = message;
  toast.className = `toast ${type}`;
  setTimeout(() => toast.className = 'toast hidden', 3000);
}

// ===== ДАШБОРД =====

async function loadDashboard() {
  const res = await fetch(`${API}/api/dashboard`);
  const d = await res.json();

  // KPI
  const kpiValues = [d.athletes_count, d.available_items, d.active_issuances, d.overdue_count, d.debtors];
  document.querySelectorAll('.kpi-value').forEach((el, i) => {
    el.textContent = kpiValues[i] ?? '-';
    el.classList.remove('skeleton');
  });

  // Категории

```

```

const maxCat = Math.max(...d.categories.map(c => c.count), 1);
document.getElementById('category-chart').innerHTML = d.categories.map(c
=> `
    <div class="chart-row">
      <span class="chart-label">${c.category}</span>
      <div class="chart-bar-wrapper">
        <div class="chart-bar" style="width:${(c.count / maxCat *
100).toFixed(0)}%"></div>
      </div>
      <span class="chart-value">${c.count}</span>
    </div>
  `).join('');

// Последние выдачи
document.getElementById('recent-issuances-tbody').innerHTML = d.re-
cent_issuances.map(r => `
  <tr class="${r.effective_status === 'OVERDUE' ? 'row-danger' : ''}">
    <td>${r.athlete_name}</td>
    <td>${r.product_name}</td>
    <td>${r.issuance_date}</td>
    <td>${r.planned_return_date || '-'}</td>
    <td><span class="badge badge-${r.effective_status.toLower-
Case()}">${r.effective_status}</span></td>
  </tr>
  `).join('');

// Топ-должники
document.getElementById('top-debtors-tbody').innerHTML = d.top_debt-
ors.map(t => `
  <tr class="row-danger">
    <td>${t.athlete_name}</td>
    <td>${t.sports_category || '-'}</td>
    <td>${t.overdue_items}</td>
    <td>${t.max_days} дн.</td>
  </tr>
  `).join('');
}

// ===== СНОПТЧМЕНЫ =====

function debounceSearch() {
  clearTimeout(searchTimeout);
  searchTimeout = setTimeout(loadAthletes, 300);
}

async function loadAthletes() {
  const search = document.getElementById('athlete-search').value;
  const debt = document.getElementById('debt-filter').value;
  const res = await fetch(`${API}/api/ath-
letes?search=${search}&has_debt=${debt}`);
  const data = await res.json();
  document.getElementById('athletes-tbody').innerHTML = data.map(a => `
    <tr class="${a.has_active_debt ? 'row-warning' : ''}">
      <td><strong>${a.full_name}</strong></td>
      <td>${a.sports_category || '-'}</td>
      <td>${a.birth_year || '-'}</td>
      <td>${a.group_name || '-'}</td>
      <td>${a.coach_name || '-'}</td>
      <td>${a.active_items}</td>
      <td>${a.has_active_debt ? '⚠️' : '✅'}</td>
      <td><button class="btn-sm" onclick="viewAthlete(${a.ath-
lete_id})">🔍</button></td>
    </tr>
  `);
}

```

```

    `).join('');
}

async function viewAthlete(id) {
    const res = await fetch(`${API}/api/athletes/${id}`);
    if (!res.ok) return showToast('Спортсмен не найден', 'error');
    const a = await res.json();
    document.getElementById('athlete-modal').classList.remove('hidden');
    document.getElementById('athlete-detail').innerHTML = `
        <h3>${a.full_name}</h3>
        <div class="detail-grid">
            <div class="detail-item"><span class="detail-label">Email</span><span>${a.email || '-'}</span></div>
            <div class="detail-item"><span class="detail-label">Телефон</span><span>${a.phone || '-'}</span></div>
            <div class="detail-item"><span class="detail-label">Разряд</span><span>${a.sports_category || '-'}</span></div>
            <div class="detail-item"><span class="detail-label">Год рождения</span><span>${a.birth_year || '-'}</span></div>
            <div class="detail-item"><span class="detail-label">Группа</span><span>${a.group_name || '-'}</span></div>
            <div class="detail-item"><span class="detail-label">Тренер</span><span>${a.coach_name || '-'}</span></div>
            <div class="detail-item"><span class="detail-label">Активных выдач</span><span>${a.active_items}</span></div>
            <div class="detail-item"><span class="detail-label">Возвращено</span><span>${a.returned_items}</span></div>
            <div class="detail-item"><span class="detail-label">Статус</span><span class="${a.has_active_debt ? 'text-danger' : 'text-success'}">${a.has_active_debt ? '⚠ Должник' : '✅ Без долгов'}</span></div>
        </div>
    `;
}

function closeAthleteModal() {
    document.getElementById('athlete-modal').classList.add('hidden');
}

// ===== ВЫДАЧИ =====

async function loadIssuances() {
    const filter = document.getElementById('issuance-filter').value;
    const res = await fetch(`${API}/api/issuance/active?filter_status=${filter}`);
    const data = await res.json();
    document.getElementById('issuances-tbody').innerHTML = data.map(d => `
        <tr class="${d.effective_status === 'OVERDUE' ? 'row-danger' : ''}">
            <td><strong>${d.athlete_name}</strong></td>
            <td>${d.product_name}</td>
            <td>${d.size || '-'}</td>
            <td>${d.issuance_date}</td>
            <td>${d.planned_return_date || '-'}</td>
            <td><span class="badge badge-${d.effective_status.toLowerCase()}>${d.effective_status === 'OVERDUE' ? `Проспечено (${d.days_overdue}д)` : 'Активна'}</span></td>
            <td><button class="btn-sm btn-return" onclick="returnInventory(${d.issuance_document_id})">↩</button></td>
        </tr>
    `).join('');
}

async function openIssueModal() {

```

```

        document.getElementById('issue-modal').classList.remove('hidden');
        const res = await fetch(`${API}/api/athletes`);
        const data = await res.json();
        document.getElementById('issue-athlete').innerHTML = '<option value="">—
Выберите —</option>' +
            data.map(a => `<option value="${a.athlete_id}">${a.full_name}
${a.has_active_debt ? '⚠' : ''}</option>`).join('');
        document.getElementById('issue-inventory').innerHTML = '<option
value="">— Сначала выберите спортсмена —</option>';
    }

function closeIssueModal() {
    document.getElementById('issue-modal').classList.add('hidden');
}

document.getElementById('issue-athlete').addEventListener('change', async
function () {
    const res = await fetch(`${API}/api/inventory/available`);
    const data = await res.json();
    document.getElementById('issue-inventory').innerHTML = '<option
value="">— Выберите —</option>' +
        data.map(i => `<option value="${i.inventory_item_id}">${i.barcode} —
${i.nomenclature_name} (${i.size || '6/p'})</option>`).join('');
    });

async function issueInventory() {
    const res = await fetch(`${API}/api/issuance/issue`, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
            athlete_id: parseInt(document.getElementById('issue-ath-
lete').value),
            inventory_item_id: parseInt(document.getElementById('issue-in-
ventory').value),
            planned_return_date: document.getElementById('issue-return-
date').value,
        }),
    });
    const data = await res.json();
    showToast(data.message, data.status);
    if (data.status === 'success') {
        closeIssueModal();
        loadDashboard();
        loadAthletes();
        loadIssuances();
        loadInventory();
    }
}

async function returnInventory(id) {
    const res = await fetch(`${API}/api/issuance/return`, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ issuance_id: id }),
    });
    const data = await res.json();
    showToast(data.message, data.status);
    loadDashboard();
    loadIssuances();
    loadInventory();
    loadAthletes();
}

// ===== Склад =====

```

```

async function loadInventory() {
  const res = await fetch(`${API}/api/inventory/available`);
  const data = await res.json();
  document.getElementById('inventory-tbody').innerHTML = data.map(i => `
    <tr>
      <td>${i.barcode}</td>
      <td>${i.nomenclature_name}</td>
      <td>${i.nomenclature_category}</td>
      <td>${i.size || '-'}</td>
    </tr>
  `).join('');
}

// ===== СТАТИСТИКА =====

async function loadStatistics() {
  const res = await fetch(`${API}/api/inventory/statistics`);
  const data = await res.json();
  document.getElementById('stats-container').innerHTML = data.map(s => `
    <div class="stat-card">
      <h3>${s.nomenclature_category}</h3>
      <div class="stat-row">
        <span>Доступно:</span><span class="stat-availa-
ble">${s.available}</span>
      </div>
      <div class="stat-row">
        <span>Выдано:</span><span class="stat-issued">${s.is-
sued}</span>
      </div>
      <div class="stat-row">
        <span>Проблемы:</span><span class="stat-problems">${s.prob-
lems}</span>
      </div>
      <div class="stat-bar">
        <div class="stat-bar-available" style="width:${(s.available /
s.total * 100).toFixed(0)}%"></div>
        <div class="stat-bar-issued" style="width:${(s.issued /
s.total * 100).toFixed(0)}%"></div>
        <div class="stat-bar-problems" style="width:${(s.problems /
s.total * 100).toFixed(0)}%"></div>
      </div>
    </div>
  `).join('');
}

// ===== ИНИЦИАЛИЗАЦИЯ =====

loadDashboard();

```


5. Результаты работы модуля

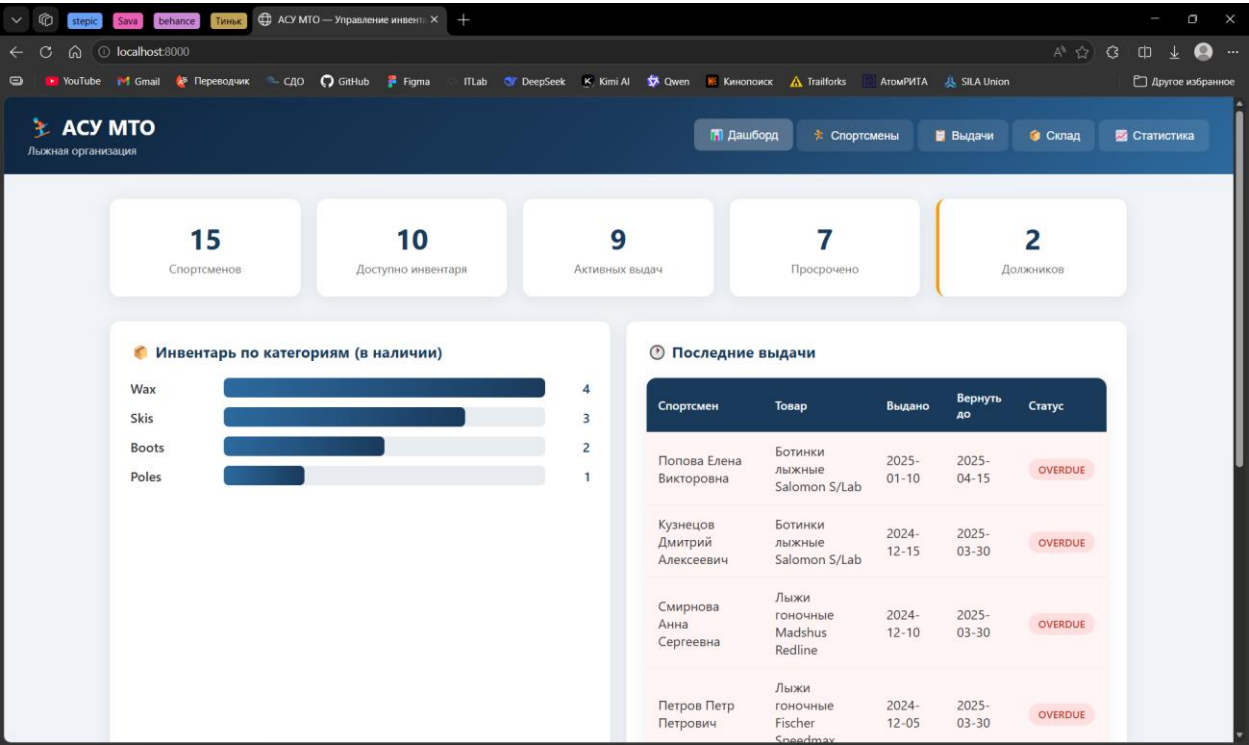


Рисунок 5.1 – Вкладка «Дашборд»

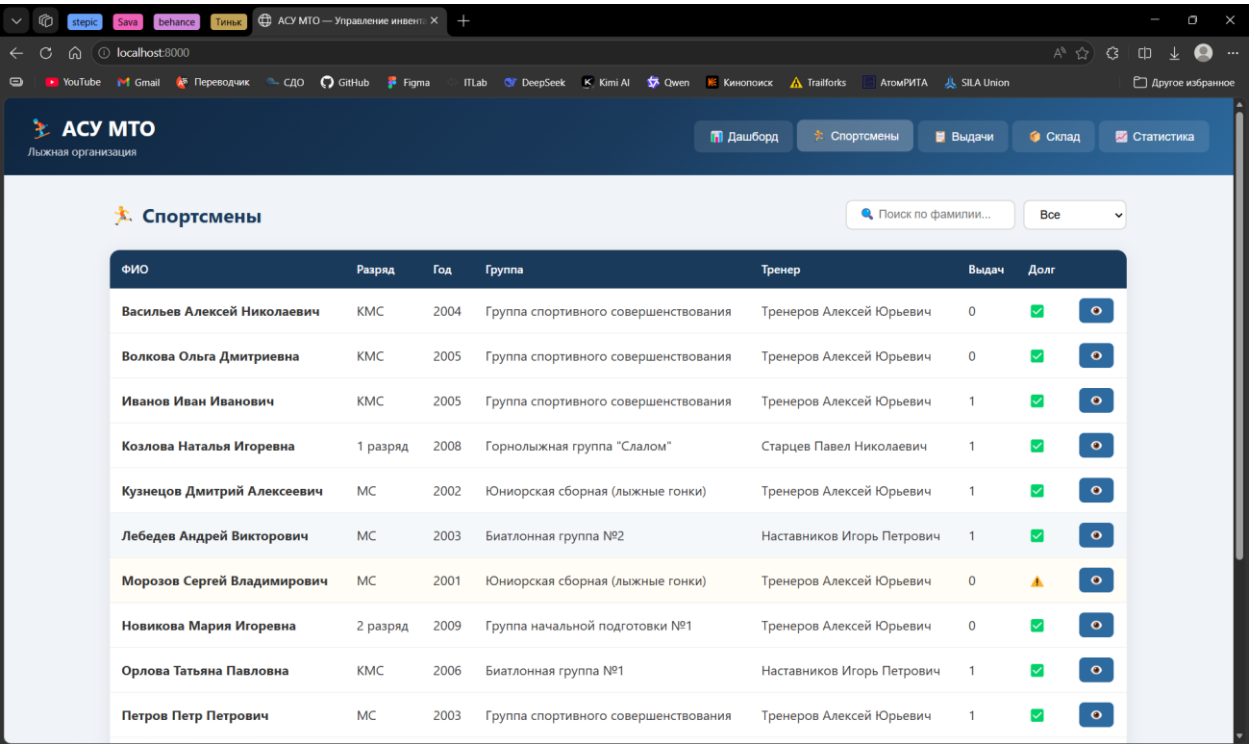


Рисунок 5.2 – Вкладка «Спортсмены»

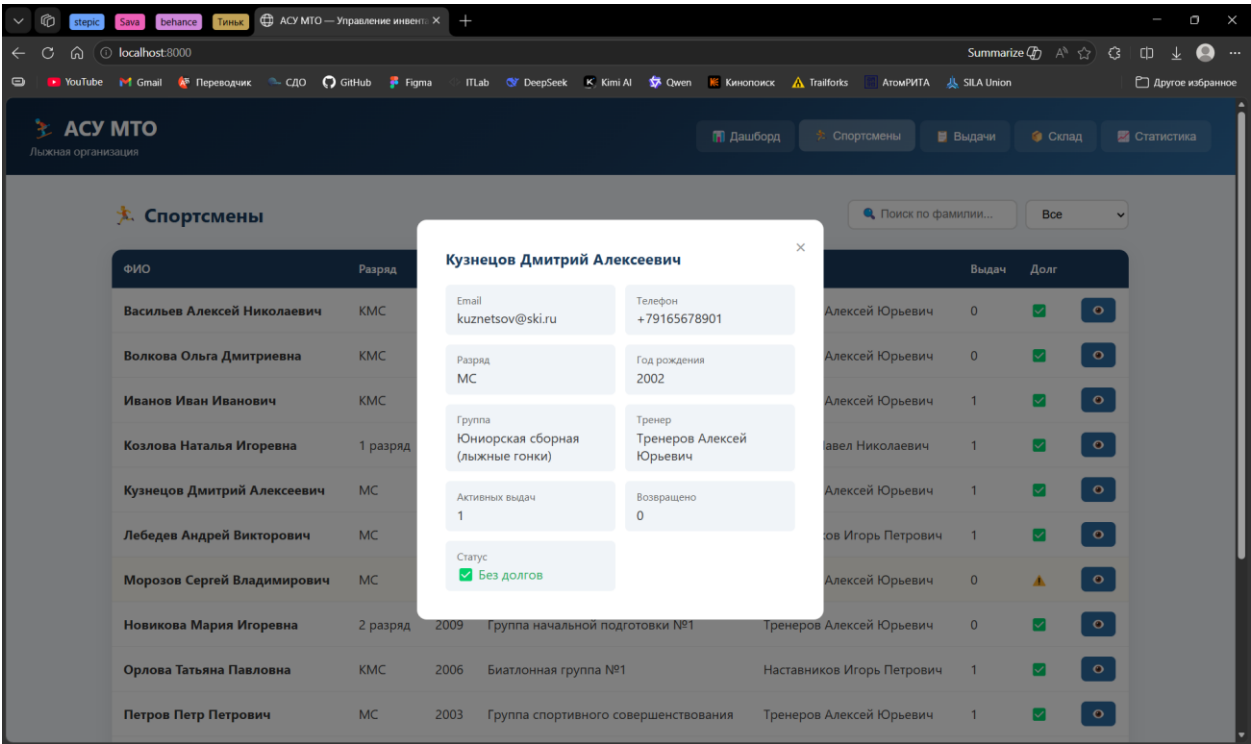


Рисунок 5.3 – Модальное окно «Детали спортсмена»

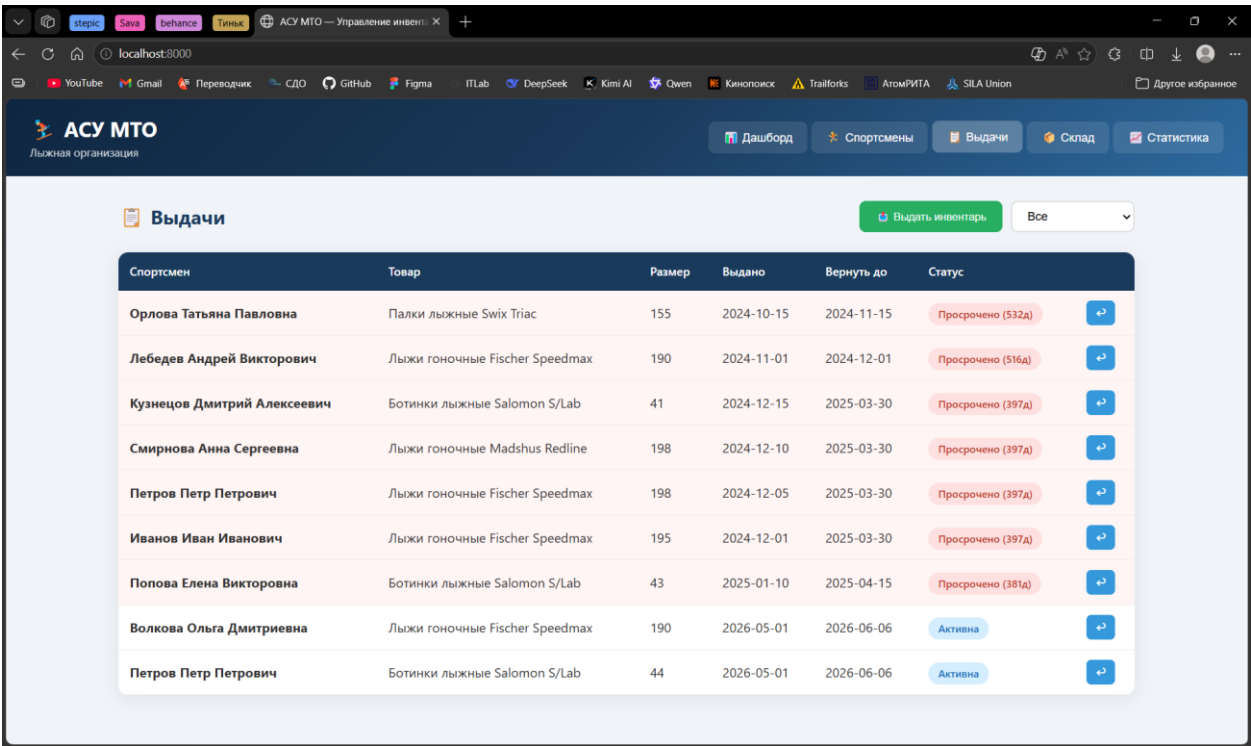


Рисунок 5.4 – Вкладка «Выдачи»

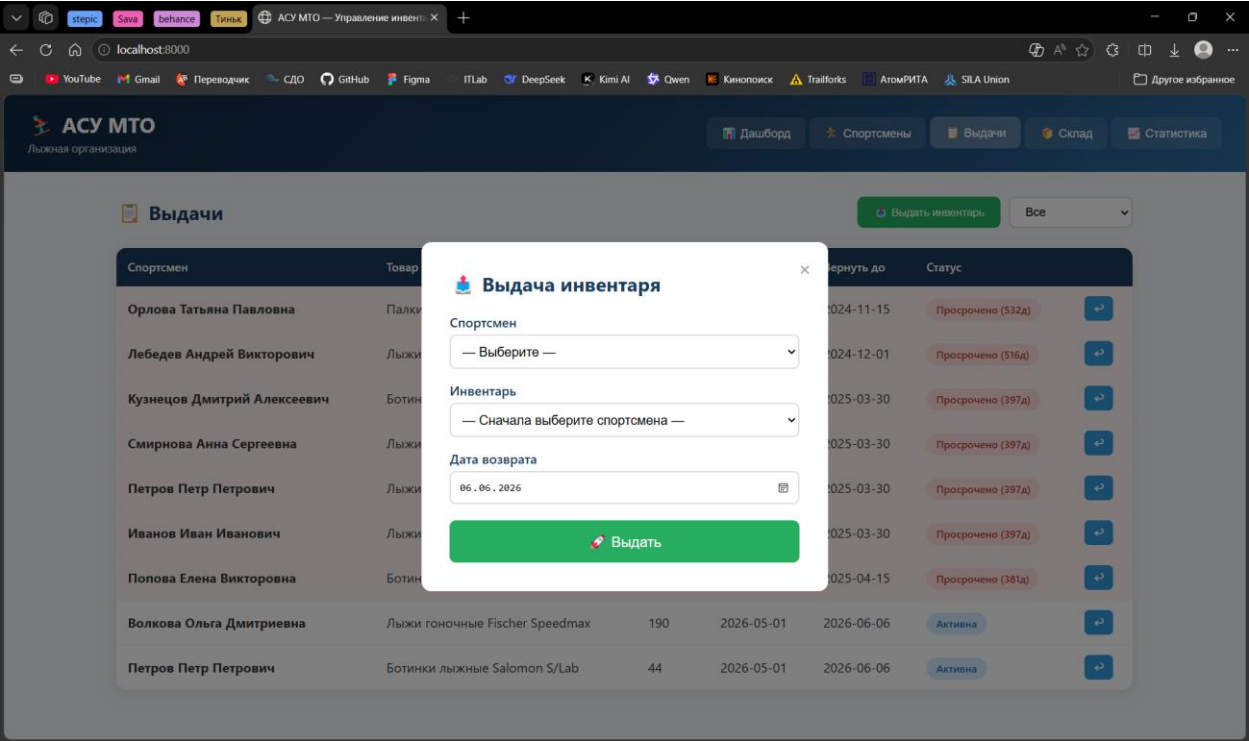


Рисунок 5.5 – Модальное окно «Выдача инвентаря»

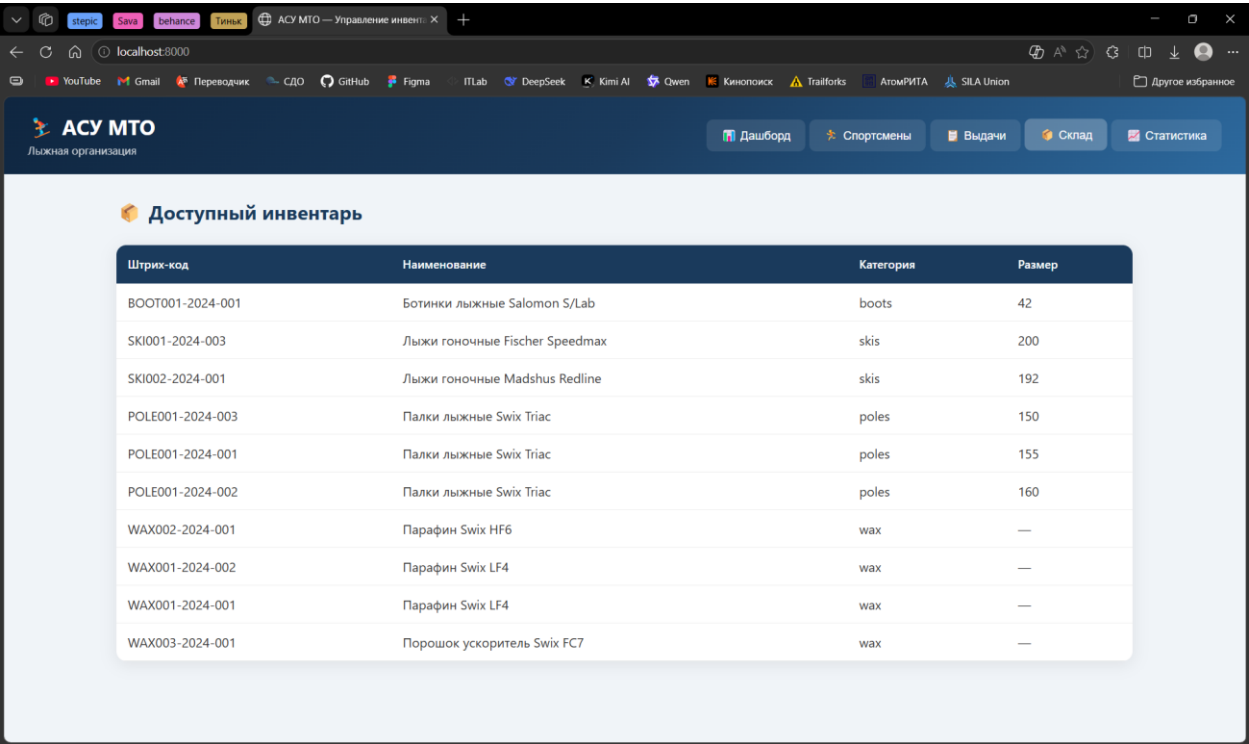


Рисунок 5.6 – Вкладка «Склад»

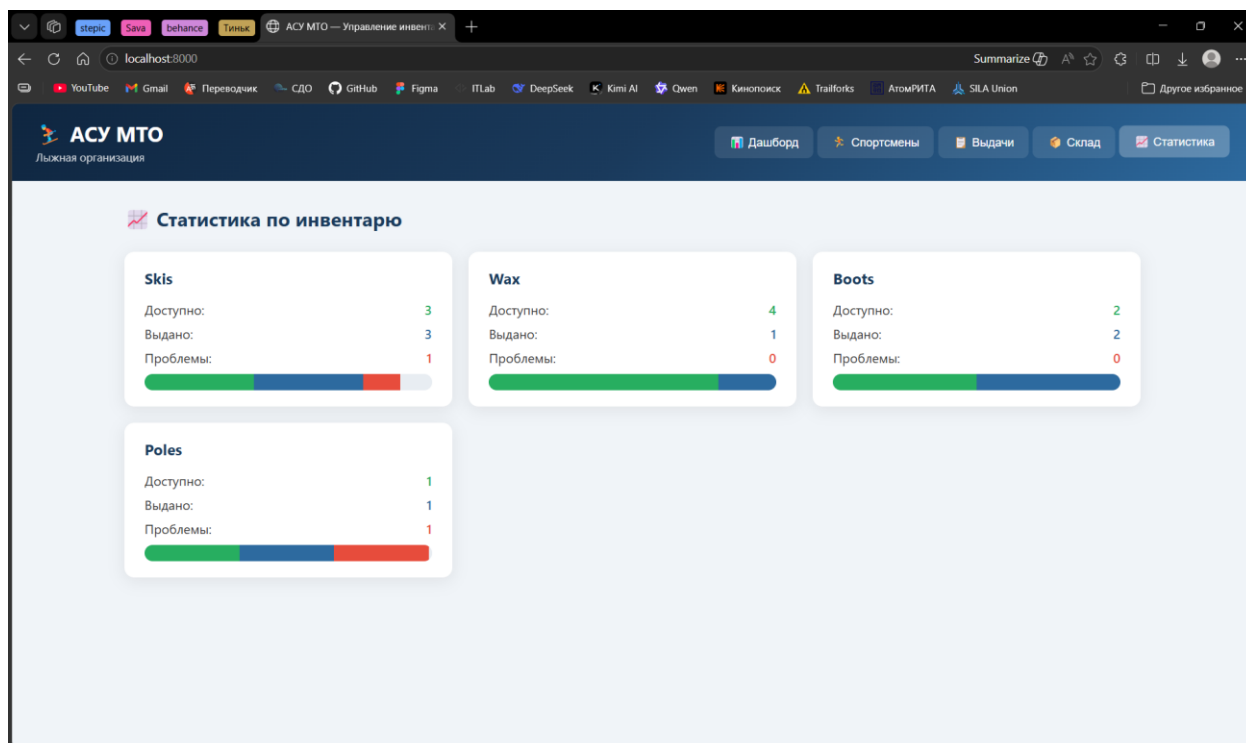


Рисунок 5.7 – Вкладка «Статистика»